

BTP-2 REPORT

Zephyr RTOS

on RPi-Pico for

Real-time applications

K P Lakshmeesh (22B1246)
EE BTech, IIT Bombay

Guides:
Prof. Siddharth Tallur
Prof. Srikanth Raj Chetupalli

INDEX

1. Introduction
2. Keyword detection application (BTP-1 continued)
3. Zephyr RTOS exploratory work
4. Work on Real-time Feasibility
5. Speech enhancement application

1. Introduction

- This report documents the work carried out during BTP-2, which is a direct continuation of BTP-1.
- In BTP-1, the main focus was on getting on-boarded with Zephyr RTOS and TFLM TinyML through a keyword detection application on RPi-Pico-W.

I understood its build system and hardware descriptions which allows portability across different boards, and also implementing TFLM pipelines.

By the end of BTP-1, I completed the foundational work for keyword detection like ADC-microphone interfacing, model training and quantization, spectrogram generation on Pico. However, I faced major roadblocks at the inference part due to a memory alignment issue.

- In BTP-2, I fixed this issue and did overall improvements to enhance accuracy and efficiency of the keyword detection application.

I started exploring other aspects which would be helpful for real-time applications and also started checking the feasibility of implementing a speech enhancement application.

In this phase I explored Symmetric Multi-Processing (SMP), Direct memory access (DMA) and data streaming from pico to PC (DMA/ADC to PC via USB), which may be very useful for several real-time applications.

I explored some models for speech enhancement and began converting and downscaling the GTCRN model to make it Pico-compatible.

2. Keyword detection application (BTP-1 continued)

The BTP-1 keyword detection pipeline uses CMSIS-DSP for MFCC feature extraction and a INT8 quantised 1D CNN TFLM model. Previously I was facing weird illogical bugs in the MFCC part of the code and sometimes during mic sampling. For eg: during spectrogram generation, it would get stuck at the 122nd frame out of the 125 frames.

I assumed the memory related bug was due to the CMSIS library, I tried creating a STFT library on my own using kissfft.

However, later on I identified the memory issue to be coming from not aligning the C-array model using "alignas(8)"

2.1 custom STFT library using kissfft

- Kissfft is a very light-weight, portable FFT library. A custom STFT wrapper was built using it (available on my github).
- Library integration:

```
zephyr_include_directories(  
    ${ZEPHYR_BASE}/../optional/modules/lib/tflite-micro/third_party_static/kissfft  
)  
  
file(GLOB app_sources src/*)  
target_sources(app PRIVATE  
    ${app_sources}  
    ${ZEPHYR_BASE}/../optional/modules/lib/tflite-micro/third_party_static/kissfft/kiss_fft.c  
    ${ZEPHYR_BASE}/../optional/modules/lib/tflite-micro/third_party_static/kissfft/tools/kiss_fftr.c  
)
```

- We have to manually add the kissfft directory using `zephyr_include_directories()`
 - This first step is needed if we are not setting `CONFIG_TENSORFLOW_LITE_MICRO=y` in `prj.conf` (for any non-ML application) which by default includes it.
- explicitly specify the required .c source files using `target_sources()`
- Memory bug:

- `kiss_fftr_alloc(256, 0, NULL, NULL)` returns a dynamically allocated pointer, which wasn't being malloced properly and failing silently.

To overcome it, I predefined the required buffers:

`kiss_fftr_alloc(256, 0, fft_cfg_buffer, &fft_cfg_buffer_len)`

The correct value of `FFT_CFG_BUFFER_LEN` was determined by first printing the size returned by `kiss_fftr_alloc` during a test run and using that as the pre-defined constant.

```
size used: 2836  
cfg address: 0x200017c8  
out address: 0x200006d0  
FFT Output:  
index 0: Real: 32.596748, Imag: 0.000000  
index 1: Real: 38.815857, Imag: -0.061893  
index 2: Real: 90.735458, Imag: -0.289242  
index 3: Real: -73.884689, Imag: 0.353082  
index 4: Real: -20.883911, Imag: 0.132956  
index 5: Real: -10.869841, Imag: 0.086409  
FFT Output:  
index 0: Real: 0.003599, Imag: 0.000000  
index 1: Real: 0.003737, Imag: 0.086650  
index 2: Real: -0.404432, Imag: -128.031555  
index 3: Real: 0.002855, Imag: -0.155571  
index 4: Real: 0.003047, Imag: -0.086439  
index 5: Real: 0.003106, Imag: -0.061723  
index 6: Real: 0.003134, Imag: -0.048582  
index 7: Real: 0.003148, Imag: -0.040381
```

-
- Output verification:
 - Few test sinusoid inputs were given. (16000 length buffer; samp_freq=16khz):
 - For 2.5 Hz, the peaks were split between bins 2,3, and DC offset shown at 0,1.
 - For 2 Hz, the peak falls fully at bin 2.

2.2 Model memory alignment fix:

- I tried again with the custom stft and a 2D CNN / stft trained model, and faced the same issue again. Noticed that spectrogram generation worked without including TFLM, and located the issue to be due to missing out on aligning the model c-array file.

```
#include "model_data_small.hpp"

alignas(8) const unsigned char model_data_small[] = {
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, 0x14, 0x00, 0x20, 0x00,
    0x1c, 0x00, 0x18, 0x00, 0x14, 0x00, 0x10, 0x00, 0x0c, 0x00, 0x00, 0x00,
```

- TFLite uses flat-buffers which are tightly packed together to save memory. It relies on parsing through the buffer using pointers relative to the base address. If the model itself is placed at an unaligned address in flash, further reads will be wrong. Adding alignas(8) solved all the weird bugs.

2.3 MFCC mismatch issue:

- The initial results on pico were inaccurate.
- The reason was that there was a scale mismatch in how TensorFlow and CMSIS-DSP calculated MFCC features. On analysing the spectrogram for the same input on both libraries, it was seen that they were off by the factor of ~ 1.33 consistently.
- This scale was adjusted on the application, and results improved noticeably.

2.4 Microphone sampling issue:

- For 16KHz sampling, we need to be able to sample at 62.5 μ s per sample.

```
for (int i=0; i<5; i++){
    start_cycles = k_cycle_get_32();
    int temp = read_mic();
    int delay = k_cyc_to_us_floor32((uint64_t)(k_cycle_get_32() - start_cycles));
    printf("delay [%d]: %d\n", i, delay);
    accu += delay;
}
```

- The above logic was used to test the time taken for read_mic() to sample. However, except for the first sample ($\sim 44\mu$ s), it showed $\sim 99\mu$ s. This was at first unexpected since I was measuring the delay only around read_mic() api and printing after it. However, the printf() was the cause of the delay, with the reasonable explanation being that it triggered a ISR which would have ran during the next mic_read() call, thereby adding latency. On removing printf's the average delay was $\sim 44\mu$ s.
- However, still there were minor fluctuations in the delays, and modified the code to busy wait dynamically until the 62.5 μ s delay is met.

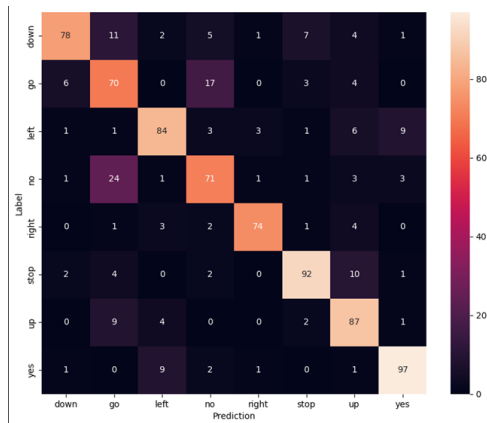
```
for (uint16_t i=0; i<iters; i+=2){
    next_deadline += period_cyc;
    out[i] = read_mic();
    while(k_cycle_get_32() < next_deadline){
    }
}
```

- Note: the keyword detection uploaded on git uses 8khz sampling for getting a larger delay window.
- DMA-based ADC sampling was identified as the best long-term solution, instead of directly using Zephyr's ADC api's. It is required to run other tasks on the CPU while reliable sampling happens in background to allow real-time keyword detection.

2.5 Further improvements:

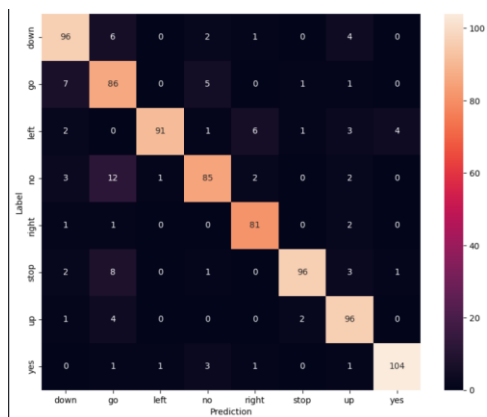
a) Model retraining

a. Older model:



```
# model for mfccs as input (1D CNN)
model = models.Sequential([
    layers.Conv1D(8, 3, activation='relu', input_shape=(125, 13)),
    layers.MaxPooling1D(2),
    layers.Conv1D(16, 3, activation='relu'),
    layers.MaxPooling1D(2),
    #layers.Dropout(0.3), # (adding dropout caused accuracy to decrease)
    layers.Flatten(),
    layers.Dense(32, activation='relu'),
    layers.Dense(num_labels, activation='softmax')
])
```

newer model:



```
#improved version using avg pooling instead of flattening + additional conv layer
model = models.Sequential([
    layers.Conv1D(16, 4, activation='relu', input_shape=(125, 13)),
    layers.MaxPooling1D(2),
    layers.Conv1D(32, 4, activation='relu'),
    layers.MaxPooling1D(2),
    layers.Conv1D(64, 4, activation='relu'),
    layers.MaxPooling1D(2),
    layers.GlobalAveragePooling1D(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(num_labels, activation='softmax')
])
```

- b. - The older model had significant accuracy issue between “no” and “go” keywords.
 - instead of flattening the conv output and directly feeding to a fully connected dense layer, which converts the entire multidimensional features into a single vector, I used `GlobalAveragePooling1D()` which reduces each feature channel to a single mean value, thereby saving space, and also reducing overfitting.
 - With the space created above, added another `Conv1D` layer, to improve model accuracy.
- c. One suggestion, was to increase the number of DCT coefficients of the spectrogram being fed to the model.

However, the accuracy didn't improve significantly, for the drastic increase in memory.

Thus, I went forward with only 13 DCT coefficients

# DCT coefficients	Validation accuracy (over 5 runs)	#params	Spectrogram space
13	~0.788	15.86KB	52KB
16	~0.786	15.9KB	64KB
20	~0.776	16.0KB	80KB
26	~0.792	16.17KB	104KB
40	~0.792	16.5KB	160KB

- d. After minimising the space allocated for the Tensor-arena, only ~22% of Pico's ram was used for the application.

b) Streaming spectrogram into tensor arena

- Initial versions used to save the MFCC output in a global array, and then feed the entire array to the model tensor arena for inference. However, this consumed a lot of SRAM.
- Now, instead we do a row-wise streaming. Each MFCC frame is computed and written directly into the tensor arena, one row at a time.
- The streaming version initially got stuck at the last MFCC frame. The cause was that MicroMutableOpResolver was declared on the stack inside a function. moving it to global scope resolved the issue.

c) Real-time feasibility

- At the default pico configuration (ie 133MHZ clock rate), below are the time taken for each task:

Task 1	Audio sampling	1 sec
Task 2	Spectrogram generation	~1.36 sec
Task 3	TFLM inference	~0.1 sec

- For real-time, we need to complete tasks 2,3 during the sampling duration. However, that's was not feasible with the original clock-rate. This issue can be overcome by increasing the clock rate to 255MHz (doubling speed) manually, as will be explained in later part of the report.

2.6 Results

```
invoke status: 0
0:'down' 1:'go' 2:'left' 3:'no' 4:'right' 5:'stop' 6:'up' 7:'yes'
predictions:
output 0: 0.101562
output 1: 0.132812
output 2: 0.000000
output 3: 0.757812
output 4: 0.000000
output 5: 0.000000
output 6: 0.000000
output 7: 0.003906
Detected keyword: no
```

(example output for keyword "no")

keyword	Inference confidence
No	~0.757
Go	~0.777
Right	~0.789
up	~0.758

The system performed well for 'go', 'no', 'right', 'up', and they were differentiated better than the older model.

However, it was hard to detect 'down', 'stop', 'left' 'yes'.

For eg, 'yes' gets detected as 'no' frequently.

The model was also only detecting it by playing the training dataset audio into the mic, suggesting possible overfittings.

3. Zephyr RTOS exploratory work

3.1 Multi-threading

Useful references: [\[1\]](#), [\[git-repo\]](#)

- We define the thread stacks and allocate required stack space

```
#define STACK_SIZE 512
K_THREAD_STACK_DEFINE(t1_stack, STACK_SIZE);
K_THREAD_STACK_DEFINE(t2_stack, STACK_SIZE);
```

- Then define the thread functions with required arguments if any. Note that the function should always be defined as the below format with the 3 void args, irrespective of using them or not.

```
void thread1_func(void *arg1, void *arg2, void *arg3){
    // size_t unused;
    while(1){
        printk("Thread 1 is running\n");
        // k_thread_stack_space_get(k_current_get(), &unused);
        // printk("Stack unused: %d bytes\n", unused);
        k_sleep(K_MSEC(1000));
    }
}

void thread2_func(void *arg1, void *arg2, void *arg3){
    while(1){
        printk("Thread 2 is running\n");
        k_sleep(K_MSEC(500));
    }
}
```

- K_thread_create() then creates the thread and puts the thread in the scheduler's ready queue. the syntax being:

```
k_tid_t k_thread_create (struct k_thread * new_thread,
                        k_thread_stack_t * stack,
                        size_t stack_size,
                        k_thread_entry_t entry,
                        void * p1,
                        void * p2,
                        void * p3,
                        int prio,
                        uint32_t options,
                        k_timeout_t delay )
```

We input any parameters through 'p1,2,3', I have set them to NULL since I don't wish to for this example. Set the thread priority using 'prio'.

options: 0 means no options are selected, thus default behaviour. These are architecture-specific, and can include K_ESSENTIAL, K_FP_REGS, and K_SSE_REGS (refer documentation for details).

delay: tells the scheduler when to tag the thread as ready. I have used K_NO_WAIT which means immediately. But we can also use other Zephyr macros like K_MSEC(1000) to start after 1 sec.

```
struct k_thread t1,t2;

int main(void){
    k_sleep(K_MSEC(5000));
    printk("Main thread started\n");
    k_thread_create(&t1, t1_stack, STACK_SIZE,
        thread1_func,
        NULL, NULL, NULL,
        1, 0, K_NO_WAIT);

    printk("T1 thread created\n");
    k_thread_create(&t2, t2_stack, STACK_SIZE,
        thread2_func,
        NULL, NULL, NULL,
        1, 0, K_NO_WAIT);
    printk("T2 thread created\n");

    while(1){
        printk("Main thread is running\n");
        k_sleep(K_MSEC(1000));
    }
}
```

```
*** Booting Zephyr OS build v4.2
Main thread started
T1 thread created
T2 thread created
Main thread is running
Thread 1 is running
Thread 2 is running
Thread 2 is running
Main thread is running
Thread 1 is running
Thread 2 is running
Thread 2 is running
```


- Key lessons learnt while trying multi-threading:
 - o No additional Kconfig entries are required other than the scheduling algorithm selection.
 - o Too small thread stack causes silent failure.
initially I had set it to 256 Bytes, increasing to 512 bytes resolved it.
 - o (this is applicable for all applications using USB console)
USB console snippet takes some time to initialise. A safe 5 second sleep delay was added at start to make the serial monitor work.

3.2 Symmetric multi-processing (SMP) in Pico

- SMP means that multiple cores are capable of running Zephyr threads. The scheduler can assign any ready thread to any free core, thus allowing truly parallel processing.
- SMP is not supported on Pico yet, and the goal was to understand why so.
- `arch_swap()` vs `arch_switch()`:
 - o Traditionally, Zephyr used `arch_swap()` for context switching. This API assumes that the scheduler has selected the next thread before the switch, ie, at each `arch_swap()` it loads the locally cached thread and also decides what thread to run after the newly loaded one. The design was inherently designed to work for single-core.
with this architecture, its possible that both core0 and core1 can independently select the same 'next thread', which is illegal.
`Z_swap()` decided whether a context switch was needed and called `arch_swap()`. Then within `arch_swap()` there were many arch-dependent assembly instructions to carry out the switch. To make this work for SMP, would mean that they would need to add cross-CPU synchronisation for all the supported architectures in assembly, which is hard to do. There were also a few other improvements which may have made it worth it for Zephyr to upgrade to `arch_switch()`.
 - o Now, they are changing to `arch_switch(to_thread, from_thread)` API for context switching, which can also support SMP. Here, the next thread decision is made right at the moment of switching, at a global scope.
Zephyr has attempted to keep the use of assembly level in `arch_switch()` to a minimum, and do most work in C by `z_swap()`. Thus, making it easier to scale it for other architectures in future, compared to the older system.
 - o Also see: (`zephyr/kernel/include/kswap.h`), (`zephyr/kernel/sched.c`), (`zephyr/kernel/smp.c`)
 - o Recently, the swap to switch migration is complete for Arm64, and RISC-V. Still left for Nios2, Arm32, Posix, x86-32 bit. Other Zephyr supported arches are already supporting `arch_switch()`.
- Status for Cortex-M:
 - o Currently the development is on-going for SMP for Cortex-M.
 - o As per a git thread [\[3\]](#), `arch_switch()` has reported 60% improvement in context switch latency compared to the older `arch_swap()`.
 - o Now, few more work is left to be done including CMSIS API migration instead of assembly, ARM TrustZone support, and few more.
 - o As per Zephyr's release roadmap, SMP support for pico's RP2040 is scheduled for October'26

- Limitation for Cortex-M0+:
Higher Cortex-M (M3 onwards) have several features which make it easier to implement SMP like atomic LDREX/STREX. Thus, the implementation for cortex-M0 is not as straightforward

References:

<https://github.com/zephyrproject-rtos/zephyr/issues/59826>

<https://github.com/zephyrproject-rtos/zephyr/issues/19932>

<https://github.com/zephyrproject-rtos/zephyr/pull/85248>

<https://github.com/zephyrproject-rtos/zephyr/pull/85080>

<https://github.com/zephyrproject-rtos/zephyr/issues/105320>

<https://github.com/orgs/zephyrproject-rtos/projects/13/views/1>

3.3 Multi-core processing in Pico

- Since zephyr SMP is not available on RP2040 yet, another approach for parallel processing was to try run core 0 on zephyr, and core 1 on Pico SDK binary.
- Approach:
 - Compile Core 1 binary using Pico SDK on PC, and save it as an array in the zephyr application.
 - From the zephyr application on core 0, write the binary address to sio->fifo_wr (see hardware/structs/sio.h) to launch it on Core1 (using the specific launch sequence as mentioned in rp2040 doc under 2.8.2 Launching Code on processor core 1)
 - For inter-core communication we can use the SIO fifo.
- Limitations:
 - Zephyr doesn't expose the Pico SDK headers since it can cause potential framework conflicts, and let to many include resolution failures.
 - Zephyr's lock implementation is software-based and Pico-SDK uses hardware-based. The two cannot inter-operate and thus, we cannot manually setup the libraries and try creating a multicore application using zephyr core0 + baremetal pico sdk core1 (atleast with dependencies on each other)
 - With the above approach, we can definitely have basic communication between the cores. However, it has to be handled carefully. For eg, printing from both cores can cause inter-weaved outputs. Instead we can assign only one core to print all data, sent via SIO fifo.

4. Work on Real-time feasibility

4.1 Direct Memory Access (DMA)

- Typically when we do an ADC read, the data is transferred in two steps. The data is first read from the ADC and written into a CPU register, then again CPU writes that register data back into the required memory location. Doing this continuously is a very inefficient use of CPU, where it could be working on other more harder tasks. Instead to make this a single step efficient transfer, we have DMA.
- DMA helps by offloading this redundant work from the CPU. The CPU can command the DMA to transfer data from one location / peripheral to another, and then the CPU can do other useful work, allowing for very high throughput and also parallel execution of work which may help for some real-time applications.

- RP2040 Zephyr DMA support:

- o These are all the modes for which the Zephyr DMA APIs support:

```
enum dma_channel_direction {  
    /** Memory to memory */  
    MEMORY_TO_MEMORY = 0x0,  
    /** Memory to peripheral */  
    MEMORY_TO_PERIPHERAL,  
    /** Peripheral to memory */  
    PERIPHERAL_TO_MEMORY,  
    /** Peripheral to peripheral */  
    PERIPHERAL_TO_PERIPHERAL,  
    /** Host to memory */  
    HOST_TO_MEMORY,  
    /** Memory to host */  
    MEMORY_TO_HOST,  
};
```

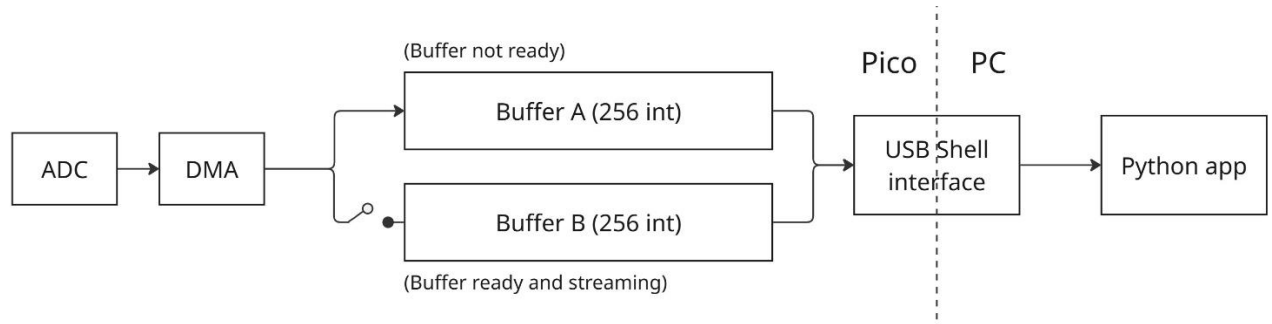
However, for pico, only M2M, M2P and P2M are supported (see dma_rpi_pico.c)

- o With DMA we can go upto 500KHz ADC rate (48Mhz/96cycles), which is a very big upgrade from using adc_read() API repeatedly (~45us per sample or 22.2 KHz)
- o The below pico specific values were used for the DMA applications created by me:
 - ADC FIFO register address (from rp2040 datasheet) = 0x4004C00C
 - This is the source address for the DMA to start the ADC transfer from.
 - DMA slot for ADC (as per zephyr API) = 0x0B
 - I had directly hardcoded the dma_slot in the dma config as 0x0b in the initial phase for quick testing. In future iteration, I will update it to make the application code meet zephyr's portability standards.

4.2 PC streaming and shell interface

- Streaming from pico (using DMA) to PC serially and having a shell interface to control actions, may be useful for many real-time applications. For example,
 - o Keyword detection and then actuating some action serially, or storing the recorded audio on PC for later analysis
 - o Speech enhancement on pico and transfer the clean audio to the PC for storing.
- I started building a demo application which streams ADC data from a microphone to PC over a shell interface, where we can control the recording to start/stop.

- Overall application flow:



- Two buffers are used. One keeps getting filled by the DMA while the other ready buffer transfers the data to PC, and after its done. Then it switches to the ready bufferA for streaming, and fills bufferB.
- Python (COM3) sends start/stop command over shell and triggers start/stop of ADC and DMA.

- Setting 2 virtual COM ports:

- I setup 2 COM ports, for convenience of debugging. One for shell commands and also receiving printk statements for debugging. Second for raw data streaming. A python script was made to manage both COM ports from PC.
- I use the zephyr snippet “cdc-acm-console” while building which creates a COM port. To create another port, we only need to define another node in the .overlay file, with the required compatible label.

```

&zephyr_udc0 {
    /* cdc-acm-console snippet already makes a port, so we make one more in addition */
    cdc_acm_lkp: cdc_acm_lkp {
        compatible = "zephyr,cdc-acm-uart";
    };
};

```

- Note regarding Baud-rate:

- the CDC-ACM ignores baud rate settings and transfers data at the full speed for USB2.0 (ie 12Mbps). The baud rate settings mentioned in the zephyr drivers are only for legacy compatibility reasons, required during USB enumeration.
- For 16KHz, 16-bit samples, we need atleast 320Kbps ($16000 \times 16 \times (10/8)$), which is well under the limit. The next standard baud rate 460800 was chosen.

- App explanation:

- We register the root shell commands using this macro:

```

SHELL_CMD_REGISTER(start, NULL, "Start recording", cmd_start);
SHELL_CMD_REGISTER(stop, NULL, "Stop recording", cmd_stop);
SHELL_CMD_REGISTER(test_usb, NULL, "Test usb streaming", test_usb_cmd);

```

I have set the sub-command arg to NULL, but can be used to add further sub-commands to a root command. [\[5\]](#) [\[6\]](#)

We enter the command specified in the first arg to do instructions defined in function at 4th arg. 3rd arg is a help string.

- Define the actions to be done by the shell commands:

```
static int cmd_start(const struct shell *shell, size_t argc, char **argv){
    ARG_UNUSED(argc);
    ARG_UNUSED(argv);
    ARG_UNUSED(shell);
    // shell_print(shell, "Recording started");
    printk("Recording started\n");
    record = true;
    dma_setup2();
    dma_start(dma_dev, dma_channel);
    adc_run(true);
    return 0;
}

static int cmd_stop(const struct shell *shell, size_t argc, char **argv){
    ARG_UNUSED(argc);
    ARG_UNUSED(argv);
    ARG_UNUSED(shell);
    // shell_print(shell, "Recording stopped");
    printk("Recording stopped\n");
    adc_run(false);
    dma_stop(dma_dev, dma_channel);

    record = false;
    buffer_a_filled = false;
    buffer_a_ready = false;
    buffer_b_ready = false;

    return 0;
}
```

- The uart_poll_out() API is used to transmit data from buffer via the console

```
void usb_send(const struct device *dev, uint8_t *data, size_t len){
    for (size_t i = 0; i < len; i++) {
        uart_poll_out(dev, data[i]);
    }
}
```

This api is blocking in nature, and busy waits for the duration of transmission.

Another API is uart_fifo_fill() which is interrupt driven. It fills the fifo as much as possible, and returns back to the CPU even if data hasn't been transmitted. So we ourselves need to keep track of bits sent so far. Uart_poll_out() was used for safety and testing.

4.3 Overclocking

- One way to overcome timing related constraint is to speed-up the microcontroller clock, to meet small improvements in deadlines.
- The default clock-rate for pico is 133MHz. I overclocked it to 250MHz, approximately doubling the computation speed, and making it feasible to make the keyword detection application real-time.
- We may also need to correspondingly increase the core voltage for stability. However, the zephyr overlay approach did not change the voltage regulator state.

A non-portable workaround is by using vreg_set_voltage() from picoSDK, if necessary.

- Enabling overclocking:

```
CONFIG_SYS_CLOCK_HW_CYCLES_PER_SEC=250000000
```

we can change the clock within reasonable ranges by setting this config option in prj.conf.

- Maximum speed-up in pico: [\[7\]](#)
 - We can get stable overclock upto 250-270 at default voltage and upto 360MHz by increasing core voltage.
- Testing:
 - Done in stft application (check github)
 - The time taken to take stft for 16k samples (256 window, 128 step) was observed at two clock rates.

Clock rate	Time taken
133Mhz	0.979 sec
250Mhz	0.489 sec

- A 2x improvement is observed from 1.88x increase in clock rate

5. Speech enhancement application

Several models available online were briefly evaluated to see which would be feasible to implement on pico with good accuracy. RNNoise and GTCRN seemed like the only two best models.

5.1 RNNoise

- RNNoise was considered at first since the INT8 quantised TFLite model was already available online, which simplifies deployment. [\[8\]](#)
- It works on 10ms audio frames. It relies heavily on both DSP and a RNN to clean out noise.
- First a biquad high pass filter is applied to remove low frequency noise and DC offset.
- Broadly, we compute the band energies for each frame and apply DCT on the log spectrum (22 bark-frequency cepstral coefficients), These can be tuned to adjust broad ranges of frequencies together, but we cannot deal with the finer details with this alone. It may suppress the noise along with the speech. Narrow windows of speech can be let through by pitch filtering (6 pitch features).
- The first and second derivative wrt time of first 6 BFCCs and pitch period a spectral non-stationarity metric (helpful in speech detection) are also included ; thus totally 42 features are given as input to the GRU model. It returns 22 gain values and are applied to each of the frequency bands, and then pitch filtering is performed.
- Then the audio is reconstructed, frame wise. It is also ensured that the gains between the frames doesn't change unnaturally.
- Adaptations tried:
 - o The pitch filter stage seemed to be computationally expensive (involved auto-correlations and many multiplications). I had a doubt whether real-time would be feasible. A possible approximation would have been to avoid pitch filtering entirely. This would bring down computation significantly at the cost of quality.
 - o Before I made DMA work, only 16Khz sampling was working so I made an effort to convert the model from 48Khz to 16Khz sampling. However, this would require readjusting the bark-scale frequency boundaries (eband5ms array) and also retrain the entire model.
- However, many part of the pre-processing C codes were specific for PC implementation. Thus, they couldn't be used directly and required re-writing them to make it MCU compatible. This would take a significant amount of time and did not seem to align with the objectives of BTP-2 and hence discontinued.

5.2 GTCRN (gated temporal convolutional recurrent network)

- On the github page, only the .onnx model was available. It was generated from an initial pytorch model script. I first made attempts to directly convert from onnx to tflite, and faced many compatibility issues.
- ONNX to TFLite conversion:
 - o Onnx2tf python library:
Faced a lot of library resolution and version dependency related issues. Then finally, faced a dynamic dimension related issue, no channel re-arrangement options made it work.
 - o Onnx and onnx-tf python library:
similarly library/version resolution issues. Dimension mismatch issue during matrix multiplication.
 - o Emmatrix onnx-to-C converter:
dimension mismatch issue from GRU units for larger model. GRU bidirectional unsupported issue for simplified gtcn model.
 - o All the above issues hinted that we needed to redesign the model, to allow the conversion to happen.

- PyTorch to Tflite conversion:
 - It was suggested to directly use LiteRT to convert from the original pytorch model to tflite.
 - It is required to run this converter on a linux environment for library availability.
 - Converter issues:
 - XLA compiler can't handle grouped convolutions (groups>1 is not supported) in ConvTranspose2d.
workaround done was to replace it with 2 single custom convolution blocks made using nn.conv2d with split inputs.
 - Error in ConvBlock() from nn.ConvTranspose2d, due to upsampling for stride>1.
Reimplemented it manually using nn.Conv2d.
 - Solving the above two issues, allowed successful conversion to tflite. The .tflite file size was 1685KB.
Now we need to fully INT8 quantise to deploy using TFLM on pico. Expected file size after successful quantisation would be ~420KB.
 - Quantisation issues:
 - Issue from Select_V2 in StreamEncoder sub-node. It's most likely from slicing of caches (conv_cache, tra_cache). Select_V2 is essentially an "if" condition
Reason to believe so is because during INT8 quantisation we are required to pass a representative dataset through the entire model to find out output ranges. But, while slicing, an "if" condition is imposed based on index value.
The model can undergo dynamic quantisation (which doesn't require representative dataset) without any problem (can be deployed on higher end microcontrollers with sufficient flash/RAM and float hardware)
 - The speculated solution is to slice the caches external to the model and feed them separately into the layers.
 - Alternative solutions:
 - Shift to the non-streaming model, which doesn't use caches. A real-time streaming version can be made by overlapping and adding frames. However, it inputs 16k samples to the model, which makes it memory expensive.
 - Cut down the width/depth of convolutions
 - Use TCN (temporal conv network) instead of Bidirectional GRU. However, it can only remember for a fixed duration.